

Project Doppel

A Distributed Privacy–Utility Benchmarking Pipeline for Synthetic Electronic Health Record Generation

Project Blueprint · Version 1.0 · 2026

1. Project Overview

Doppel is a reproducible, containerized pipeline for generating and evaluating privacy-preserving synthetic Electronic Health Record (EHR) data. The project trains three families of synthetic-data generators — a statistical baseline, a GAN-family model, and a diffusion-based model — on a public clinical dataset, then benchmarks each on three competing axes: statistical fidelity, downstream predictive utility, and resistance to privacy attacks. The entire pipeline is orchestrated as a set of Docker containers and Kubernetes Jobs, runnable end-to-end on a local cluster (minikube/kind).

Synthetic health data generation is an active and increasingly standardized research area — established architectures (CTGAN, TVAE, diffusion models) and evaluation practices (fidelity, utility, and membership/attribute inference metrics) already exist, and benchmarking suites such as SynthEHRella have begun to unify them. This project does not claim to invent a new generative architecture. Its contribution is a rigorously reproducible, distributed evaluation pipeline — something the existing literature explicitly identifies as inconsistent across studies — applied to a case the literature flags as still difficult: high-dimensional, rare clinical codes.

2. Problem Statement & Motivation

Hospitals and research institutions increasingly want to share patient data for research without violating patient privacy. Synthetic data generation offers a path forward, but current evaluation practice in the field is fragmented: studies report fidelity, utility, and privacy metrics inconsistently, using different attack models and different downstream tasks, making it difficult to compare methods on equal footing. Additionally, generative methods still struggle to faithfully reproduce rare, high-dimensional clinical code combinations.

Doppel addresses this by fixing one dataset, one evaluation protocol, and one deployment environment, then comparing three generator families under identical conditions — producing a single, reproducible fidelity–utility–privacy trade-off curve rather than three papers' worth of incomparable numbers.

3. System Architecture

The pipeline consists of four sequential stages. Stages 2 and 3 run as parallel Kubernetes Jobs — one per generator/evaluation combination — so all three methods are trained and evaluated under identical containerized conditions.

Pipeline Stage	Function	Primary Owner
1. Data Prep	Ingest MIMIC-III Clinical Database (Demo), clean and structure into tabular records (demographics, ICD-9 codes, labs, admissions).	Track 3
2. Generation	Three parallel K8s Jobs: statistical baseline, CTGAN/TVAE, and a tabular diffusion model — each producing a synthetic dataset.	Track 1
3. Evaluation	Parallel evaluation of each synthetic dataset on fidelity (JSD, correlation), utility (train-on-synthetic / test-on-real), and privacy (membership & attribute inference).	Track 2

Pipeline Stage	Function	Primary Owner
4. Aggregation	Collects results from all evaluation pods; renders the fidelity–utility–privacy Pareto frontier on a dashboard.	Track 4

4. Track Ownership

Work is organized into four tracks. Track 1 owns the project's core technical contribution — the generative modeling work, including the diffusion-based model, which is the most open-ended and iterative piece of the project. Track 2 owns the evaluation methodology. Tracks 3 and 4 own the data foundation and the systems/delivery layer that the other two tracks depend on.

Track	Owner	Core Responsibility
Track 1	Team Member A	Generative Modeling Core — design, implementation, and tuning of all three synthetic-data generators (statistical baseline, GAN-family, diffusion-based); owns the project's central technical contribution and cross-track architecture decisions.
Track 2	Team Member B	Privacy & Utility Evaluation — implementation of fidelity, downstream-utility, and privacy-attack (membership/attribute inference) modules; defines and runs the comparative evaluation protocol.
Track 3	Team Member C	Data Engineering — dataset acquisition and preprocessing, schema design, feature engineering, data validation, and interface contracts between pipeline stages.
Track 4	Team Member D	Systems Integration & Delivery — containerization, Kubernetes orchestration, results dashboard, documentation, and final delivery artifacts (demo, report, presentation).

5. Phase-Wise Execution Plan

Phase 1 — Foundation & Design (Weeks 1–2)

Track	Key Tasks
Track 1	– Literature review of generative architectures for tabular/EHR data (CTGAN, TVAE, tabular diffusion models). – Finalize model design and architecture for the diffusion-based generator — the project's core technical contribution. – Define the overall generator interface (input schema, output format) that all three models will conform to.
Track 2	– Define the evaluation protocol: fidelity, utility, and privacy metrics, with exact formulas and pass/fail thresholds. – Select the downstream utility task (e.g., 30-day readmission or in-hospital mortality prediction). – Survey membership-inference attack methodology (shadow-model approach) to be implemented in Phase 2–3.
Track 3	– Acquire and clean the MIMIC-III Clinical Database Demo dataset. – Define the data schema, feature dictionary, and train/held-out evaluation splits. – Document data access and preprocessing steps for the team wiki.
Track 4	– Set up the shared GitHub repository, branch strategy, and PR template. – Stand up the local Kubernetes development environment (minikube/kind) and base Docker images. – Draft integration contracts: schema formats and file paths agreed between all four tracks before code is written.

Phase 2 — Core Development (Weeks 3–7)

Track	Key Tasks
Track 1	– Implement and train the statistical baseline and CTGAN/TVAE generators. – Begin implementation of the diffusion-based generator (architecture + training loop) — the most iterative and open-ended component of the project. – Run initial hyperparameter sweeps across all three generators.
Track 2	– Implement fidelity metrics: Jensen–Shannon divergence, pairwise correlation preservation, dimension-wise distribution checks. – Build the downstream utility pipeline (train-on-synthetic / test-on-real classifier). – Begin implementation of the membership-inference attack.
Track 3	– Finalize the preprocessing stage as a Kubernetes Job. – Build data validation checks (schema conformance, missing-value handling). – Support Tracks 1 and 2 with data-access utilities as generator prototypes come online.
Track 4	– Containerize each generator prototype as it becomes available. – Build Kubernetes Job templates for the generation stage. – Scaffold the results dashboard (Streamlit or FastAPI + a simple front end).

Phase 3 — Integration & Evaluation (Weeks 8–11)

Track	Key Tasks
Track 1	– Finalize diffusion-generator training and tune all three generators for the final comparison run.

Track	Key Tasks
	– Support integration debugging for the generator containers inside the full pipeline.
Track 2	– Complete the privacy-attack suite (membership + attribute inference). – Run the full evaluation across all three generators. – Compute and validate the fidelity–utility–privacy Pareto frontier.
Track 3	– Validate data flow across the full pipeline end-to-end. – Manage evaluation-dataset versioning. – Assist with result-data collection and formatting for the dashboard.
Track 4	– Wire the complete multi-stage Kubernetes pipeline (prep → generate → evaluate → aggregate). – Finalize the dashboard to visualize the Pareto frontier results. – Run full end-to-end pipeline tests on minikube and fix orchestration issues.

Phase 4 — Finalization & Delivery (Weeks 12–14)

Track	Key Tasks
Track 1	– Lead the Methodology and Results sections of the project report — model design, training setup, and comparative findings. – Finalize architecture documentation for the generator components.
Track 2	– Write the Evaluation section of the report. – Prepare final metric tables and result plots for the Pareto frontier.
Track 3	– Prepare the Data & Methodology appendix. – Support final report formatting and citation checks.
Track 4	– Record the demo video and polish the GitHub README. – Prepare the course presentation slides. – Coordinate final report assembly and submission.

6. Deliverables by Phase

Phase	Deliverable
Phase 1 (Wks 1–2)	Cleaned dataset, documented schema, local K8s dev environment, integration contracts.
Phase 2 (Wks 3–7)	Three working generators (containerized), fidelity/utility metric code, dashboard scaffold.
Phase 3 (Wks 8–11)	Full orchestrated pipeline on Kubernetes, complete privacy-attack suite, Pareto frontier results.
Phase 4 (Wks 12–14)	Final report, demo video, presentation, polished repository.

7. Technology Stack

Category	Tools
Data	MIMIC-III Clinical Database (Demo), Pandas, NumPy
Generative Models	PyTorch, SDV (CTGAN, TVAE), custom tabular diffusion model

Category	Tools
Evaluation	Scikit-learn, SciPy (statistical tests), custom membership/attribute-inference attack code
Orchestration	Docker, Kubernetes (minikube/kind), Kubernetes Jobs
Dashboard	Streamlit or FastAPI + lightweight front end
Delivery	GitHub, Markdown/Word report, slide deck

8. Success Criteria

- Fidelity: synthetic feature distributions statistically close to real data (low Jensen–Shannon divergence) across all three generators.
- Utility: a model trained on synthetic data approaches the performance of a model trained on real data for the chosen downstream task.
- Privacy: membership-inference attack AUROC close to 0.50 (random guessing) for the privacy-preserving generator(s).
- Systems: the full pipeline runs end-to-end on a local Kubernetes cluster without manual intervention between stages.
- Deliverables: a working demo, a reproducible GitHub repository, and a written report presenting the fidelity–utility–privacy trade-off across methods.